SQL Indexes for Query Optimization — Interview Notes

1. What is an Index?

An **index is a database object that improves the speed of data retrieval** by creating a faster lookup path to rows.

Think of it like a **book index**:

- Without an index → Read every page (Table Scan)
- With an index → Jump directly to the required page (Index Seek)

Example:

Without index:

```
SELECT *
FROM Customers
WHERE CustomerID = 100;
```

Database may scan millions of rows.

With index:

```
CREATE INDEX idx_customer_id
ON Customers(CustomerID);
```

Database can quickly locate CustomerID = 100.

2. Why Do We Need Indexes?

Indexes improve:

✅ SELECT queries
✅ WHERE filtering
✅ JOIN performance
✅ ORDER BY sorting
✅ GROUP BY operations

Example:

```
SELECT *
FROM Orders
WHERE CustomerID = 500;
```

Good candidate:

```
CREATE INDEX idx_orders_customer
ON Orders(CustomerID);
```

3. How Does an Index Work?

Most databases use **B-tree structures**.

Without index:

```
Table

Row 1
Row 2
Row 3
Row 4
...
Row 1,000,000
```

Database checks rows one by one.

With index:

```
Index

CustomerID      Location
100             Row 500
200             Row 900
300             Row 1200
```

Database directly goes to the data.

4. Clustered Index

Definition

A clustered index determines the **physical order of data in the table**.

The table data itself is stored in the index structure.

Example:

```
CREATE CLUSTERED INDEX idx_order_id
ON Orders(OrderID);
```

Characteristics

✅ Only one clustered index per table
✅ Very fast for range searches
✅ Usually created on Primary Key

Example:

```
WHERE OrderID BETWEEN 1000 AND 2000
```

is fast.

5. Non-Clustered Index

Definition

A separate structure that stores:

- Indexed column value
- Pointer to actual row

Example:

```
CREATE NONCLUSTERED INDEX idx_customer_email
ON Customers(Email);
```

Characteristics

✅ Multiple allowed per table
✅ Does not change table order
✅ Good for search columns

Example:

```
SELECT *
FROM Customers
WHERE Email='abc@test.com';
```

6. Clustered vs Non-Clustered Index

Feature	Clustered	Non-Clustered
Stores actual data	Yes	No
Number allowed	One	Many
Faster for range queries	Yes	Sometimes
Uses pointer	No	Yes

Common use	Primary key	Search columns

7. Index Seek vs Index Scan

Index Seek ⭐ (Good)

Database directly finds required rows.

Example:

```
WHERE CustomerID=100
```

Performance:

```
Index → Required Rows
```

Index Scan ❌ (Slower)

Database reads the entire index.

Example:

```
WHERE YEAR(OrderDate)=2025
```

Database may scan everything.

8. Columns That Should Be Indexed

Create indexes on:

1. WHERE columns

Example:

```
WHERE CustomerID=10
```

Index:

```
(CustomerID)
```

2. JOIN columns

Example:

```
Orders.CustomerID =
Customers.CustomerID
```

Index:

```
Orders(CustomerID)
```

3. ORDER BY columns

Example:

```
ORDER BY OrderDate DESC
```

Index:

```
(OrderDate)
```

4. GROUP BY columns

Example:

```
GROUP BY ProductCategory
```

Index:

```
(ProductCategory)
```

9. Composite Index (Multi-column Index)

An index containing multiple columns.

Example:

Query:

```
SELECT *
FROM Orders
WHERE CustomerID=10
AND OrderDate='2026-01-01';
```

Better index:

```
CREATE INDEX idx_customer_date
ON Orders(CustomerID, OrderDate);
```

Column Order Matters

Index:

```
(CustomerID, OrderDate)
```

Works well:

```
WHERE CustomerID=10
```

Works well:

```
WHERE CustomerID=10
AND OrderDate='2026-01-01'
```

May not work efficiently:

```
WHERE OrderDate='2026-01-01'
```

because CustomerID is the first column.

10. Covering Index

A covering index contains all columns needed by a query.

Example query:

```
SELECT CustomerID, OrderDate, TotalAmount
FROM Orders
WHERE CustomerID=100;
```

Create:

```
CREATE INDEX idx_covering
ON Orders(CustomerID)
INCLUDE(OrderDate, TotalAmount);
```

Benefits:

- Database does not need to read the table
- Faster query execution

11. Index and JOIN Optimization

Slow:

```
SELECT *
FROM Orders o
JOIN Customers c
ON o.CustomerID=c.CustomerID;
```

Add indexes:

```
CREATE INDEX idx_orders_customer
ON Orders(CustomerID);

CREATE INDEX idx_customer_id
ON Customers(CustomerID);
```

Result:

Faster join matching.

12. When NOT to Use Indexes

Avoid excessive indexes.

Indexes have costs:

❌ Require storage
❌ Slow INSERT
❌ Slow UPDATE
❌ Slow DELETE

Example:

A table with 20 indexes:

- SELECT becomes faster
- But every insert must update all indexes

13. Indexing and NULL Values

Queries:

```
WHERE Email IS NULL
```

may benefit from indexes depending on database engine.

14. Functions Can Prevent Index Usage

Bad:

```
SELECT *
FROM Customers
WHERE LOWER(Name)='john';
```

Why?

Database cannot efficiently use index.

Better:

Store normalized data or use appropriate indexing strategies.

15. Index Maintenance

Indexes become fragmented.

Operations:

- Rebuild index
- Reorganize index
- Update statistics

Example:

```
ALTER INDEX idx_name
REBUILD;
```

16. Interview Scenario

Question:

"Your query takes 30 seconds. How would you optimize it?"

Answer:

1. Check execution plan.
2. Identify table scans.
3. Check missing indexes.
4. Review WHERE and JOIN conditions.
5. Remove SELECT *.
6. Rewrite non-SARGable conditions.
7. Add appropriate indexes.
8. Test performance before and after.

17. Common Interview Questions

Q: Why not create indexes on every column?

Answer:
Indexes improve reads but slow down inserts, updates, and deletes because indexes must also be maintained.

Q: Which columns are good candidates for indexes?

Answer:
Columns frequently used in WHERE clauses, JOIN conditions, ORDER BY, and GROUP BY.

Q: What is a covering index?

Answer:
An index that contains all columns required by a query, allowing the database to retrieve data without accessing the main table.

Q: Difference between index scan and index seek?

Seek:
Finds specific rows → faster.

Scan:
Reads many/all rows → slower.

Q: Why is a composite index order important?

Because databases use the **leftmost prefix rule**. The first column in the index should usually be the most commonly filtered column.

Quick Memory Rule

Index WHERE + JOIN + ORDER BY columns.